

Giving the application memory

A practical record of V3: PostgreSQL, persistent application data, connection pooling, analytics, deployment history, and database-aware tracing.

V3 DEFINITION A real PostgreSQL-backed application layer running locally in Docker, used by FastAPI for durable state and analytics, observed through the same OpenTelemetry + Alloy + Grafana Cloud pipeline established in V2.

V3 LOCAL APPLICATION LAB

Browser / Next.js

|

v

FastAPI

|

|

|

v

v

v

Postgres

Resend

OTel

|

|

+-----+

|

Alloy

|

v

Grafana Cloud -> Tempo + Application Observability

Build -> Connect -> Persist -> Instrument -> Observe -> Document

V3 keeps the production frontend on Cloudflare Pages. FastAPI, Alloy, and PostgreSQL are real and working, but remain local-only until the later AWS production phase.

V3 at a glance

Concept	In this project
Application	Next.js + React + TypeScript frontend; FastAPI backend
Data layer	PostgreSQL 18.6 running as a Docker Compose service
Database driver	psycopg with a connection pool; no ORM
Persistence	Named Docker volume mounted into PostgreSQL
Application data	Contact history, resume downloads, project views, deployment history
Observability	OpenTelemetry -> Alloy -> Grafana Cloud; PostgreSQL spans included
Email	Resend with Cloudflare Email Routing behind hello@aldhairmartinez.com
Production state	Frontend live on Cloudflare; backend/data stack still local

KEY MENTAL MODEL V2 gave the application a server and telemetry. V3 gives that server durable memory. PostgreSQL is not the frontend and it is not the email provider; it is the system of record for application state and small event histories.

The four V3 tables are intentionally independent event/fact logs. There are no foreign keys between them because each feature can stand alone and the project metadata already lives in the application/source tree.

The architecture: production vs. local

A major V3 lesson is learning to separate what is live for visitors from what is live only in the development lab.

PRODUCTION TODAY

Browser -> Cloudflare Pages -> static Next.js -> Faro -> Grafana Cloud

LOCAL APPLICATION LAB

```
Browser / Next.js
  |
  v
FastAPI
|----- PostgreSQL
|----- Resend
|----- OpenTelemetry -> Alloy -> Grafana Cloud
```

IMPORTANT Pushing the V3 code to GitHub does not put FastAPI or PostgreSQL into production. The production site is still the static Cloudflare Pages frontend.

The later AWS phase is where the local FastAPI, Alloy, and PostgreSQL design gets a real production home. That phase will also unlock real visitor analytics and a GitHub Actions deployment-recording workflow.

How PostgreSQL fits

PostgreSQL is the application database. FastAPI talks to it over the Docker Compose network; PostgreSQL does not sit inside the FastAPI container.

Layer	What it does	How this project uses it
FastAPI	Runs request-time application logic	Validates requests and decides what gets stored
psycopg	Python driver for PostgreSQL	Opens connections and executes parameterized SQL
Connection pool	Reuses database connections	Avoids opening a brand-new TCP/auth handshake for every request
PostgreSQL	Persists rows and executes SQL	Stores the four V3 tables
Docker volume	Persists database files outside container lifecycle	Keeps rows through restarts/recreation

Docker Compose DNS is the simple connection trick. A service named postgres is reachable from FastAPI as postgres:5432. Port 5432 is published to the Mac as a convenience for local tools, but container-to-container traffic does not require that host port.

A named volume matters because containers are disposable. The container can be removed and recreated while the PostgreSQL data directory survives in the volume.

The V3 database schema

The schema is intentionally boring: BIGSERIAL identifiers, TIMESTAMPTZ timestamps, clear columns, and indexes only on the time-ordered reads the application actually needs.

Table	Purpose	Core fields
contact_submissions	Durable contact history and email outcome	id, name, email, message, email_sent, email_error, created_at
resume_downloads	Anonymous download counter with timestamp	id, file_type, downloaded_at
project_views	Anonymous project-view events	id, project_slug, viewed_at
deployments	Release/deployment history	id, version, commit_sha, environment, notes, deployed_at

NOT STORED No IP addresses, user-agent strings, device/session fingerprints, geolocation, or similar tracking data. Resume and project analytics remain intentionally anonymous.

There are no foreign keys between the four tables. That is deliberate: these are independent application facts rather than a relational object graph. The project list comes from source-controlled MDX, so project_slug is validated at the application layer.

The schema is installed idempotently with CREATE TABLE IF NOT EXISTS at backend startup. This is intentionally lighter than a formal migration framework for one person and a local lab. A real multi-environment upgrade path is a later concern.

Connections, pooling, and persistence

Early V3 used a single fresh database connection for a diagnostic endpoint. Once four features started reading and writing PostgreSQL, a connection pool became the sensible next step.

```
FastAPI startup
|
+--> create psycopg ConnectionPool
|
request -> borrow connection -> execute SQL -> return connection
|
shutdown -> close pool
```

The pool is not an optimization we invented because it sounded advanced. It solves a concrete problem: repeated request traffic should not pay the full connection setup/authentication cost every time.

We also deliberately tested a harsher failure than a normal restart by removing the PostgreSQL container while FastAPI stayed up. A small number of health requests returned 503 while the pool detected stale connections, reconnected, and recovered. Subsequent requests succeeded.

LESSON A healthy system is not one that never fails. A useful application should detect stale state, recover when it can, and make the failure visible instead of silently pretending nothing happened.

The named PostgreSQL volume was also tested through container restart/removal and recreation. Data survived because the database files live in the Docker-managed volume, not in the disposable container filesystem.

The four application flows

V3 turns PostgreSQL into real application state, not a sidecar demo. Each feature uses a small write/read path behind FastAPI.

1. Contact history

```
POST /api/contact
-> INSERT contact_submissions
-> attempt Resend
-> UPDATE contact_submissions with email_sent / email_error
-> return the existing success/failure contract
```

2. Resume analytics

```
Resume button click
-> POST /api/analytics/resume-download
-> file_type + timestamp stored
-> native browser download proceeds
```

3. Project analytics

```
Project view / interaction
-> POST /api/analytics/project-view
-> validated project_slug + timestamp stored
-> aggregate counts returned to observability views
```

4. Deployment history

```
Seed real Git history now
-> GET /api/deployments for display
-> POST /api/deployments protected by X-Deploy-Token
-> future GitHub Actions can automate the write in production
```

Contact history and failure semantics

The contact flow has one important design decision: store the attempt before calling the external email provider.

Why that order matters:

If PostgreSQL records the submission first and Resend fails, the application still has a durable record that the visitor submitted something. The row can be inspected or retried later. If the email were sent first and the database write failed, there would be no record of the event, which is the worse failure for a history feature.

The code intentionally does not hold a database transaction open while waiting on the external Resend HTTP request. They are separate operations because Resend is outside PostgreSQL and cannot participate in one atomic database transaction.

Scenario	Behavior
Postgres insert succeeds, Resend succeeds	Row is marked sent; visitor receives the normal success response
Postgres insert succeeds, Resend fails	Row remains durable with email_error; visitor sees email failure according to the existing contract
Postgres unavailable	Application cannot persist the history record; failure is surfaced rather than silently discarded
Repeated submission	Each submission is an event and gets its own row; no artificial uniqueness rule is imposed

BOUNDARY The visitor does not get raw contact-submission rows over HTTP. The database is kept behind FastAPI and can be inspected locally with psql.

Anonymous analytics and deployment history

Resume and project analytics are intentionally simple counters, not a replacement for a full analytics platform. They record what happened and when, not who the visitor was.

Public API surface

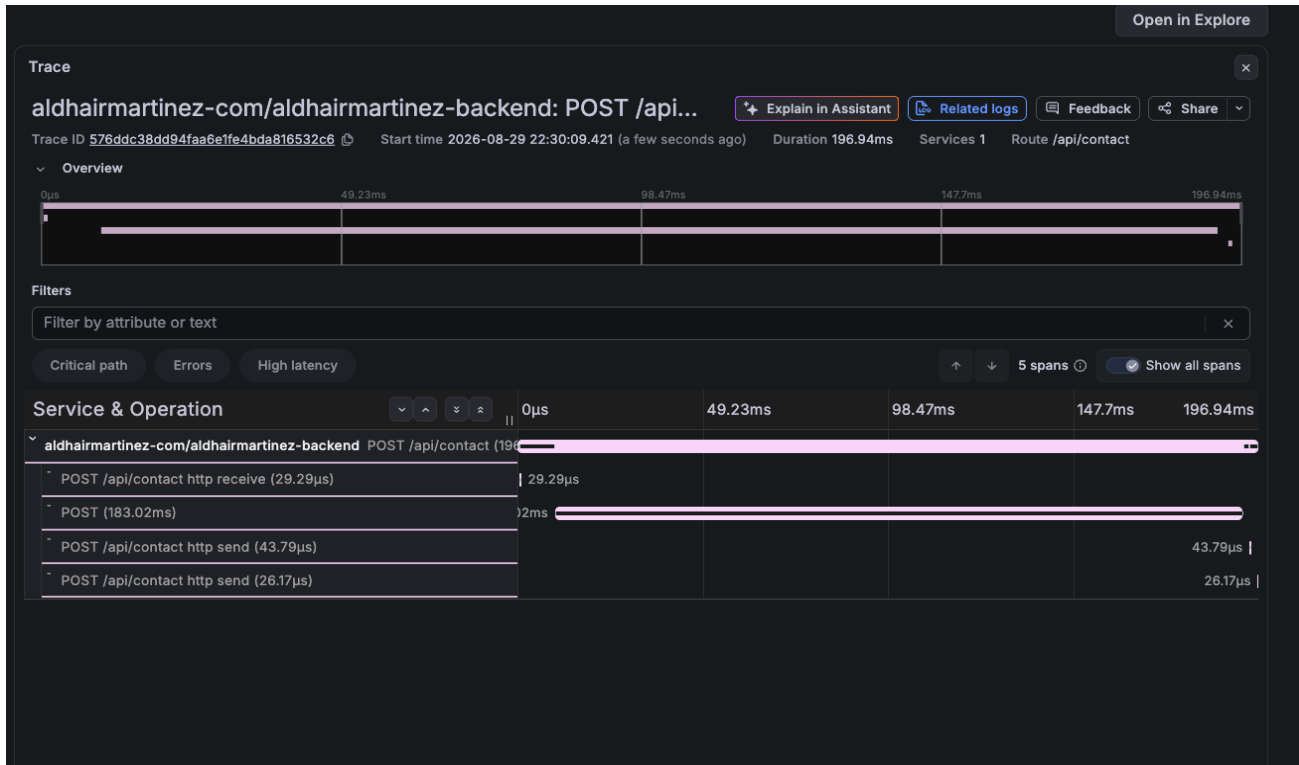
Endpoint	Access	Purpose
POST /api/analytics/resume-download	Public, no auth	Record anonymous PDF/DOCX downloads
POST /api/analytics/project-view	Public, no auth	Record validated project views
GET /api/analytics/summary	Public, aggregate only	Return counts suitable for /observability
GET /api/deployments	Public	Feed the deployment timeline
POST /api/deployments	Protected by X-Deploy-Token	Write deployment records for CI/CD

The deployment history is initially seeded from real Git history instead of fake rows. That makes the timeline a historical record of actual revisions. The future production design is to have GitHub Actions call the protected POST endpoint automatically whenever the application is deployed.

PRODUCTION CAVEAT These analytics writes are effectively local-only today because the production site has no reachable production FastAPI backend. The data layer is real; production traffic simply has nowhere to send those events yet.

OpenTelemetry: PostgreSQL inside the trace

V3 keeps the OpenTelemetry architecture established in V2 and adds automatic database spans through pycopg instrumentation.



Real contact-form trace captured in Grafana Cloud. One trace contains browser, FastAPI, PostgreSQL, and Resend work.

What this trace proves:

`POST /api/contact` is the root application request. Inside the FastAPI span are a PostgreSQL INSERT, the Resend HTTP POST, and a PostgreSQL UPDATE. The browser and backend share the same trace ID, so the dependency chain is visible as one distributed trace.

This is the right level for application tracing: it answers what did this request do, and where did the time go? It does not replace deeper database monitoring.

GOLD-STANDARD SPLIT OpenTelemetry database spans are the tracing layer. Database Observability is a separate monitoring layer we intentionally deferred. They answer different questions and complement each other.

Application Observability service map

Application Observability now recognizes the browser/frontend service and the FastAPI backend as separate services, with the backend carrying the database-aware traces.



Service map observed during V3 validation.

The important distinction is that the Service Map is a high-level application relationship view. PostgreSQL work is easier to understand from an individual trace, where the actual INSERT/UPDATE/SELECT spans are visible with timings.

For this application, the most useful chain is not “database node looks healthy.” It is “this specific request spent almost all of its time waiting on Resend while its PostgreSQL work stayed under a few milliseconds.”

What the trace taught us about performance

A representative contact submission completed in about 190 ms. The two PostgreSQL operations were tiny compared with the external email call.

Operation	Observed time	Interpretation
FastAPI request	~175.9 ms	Most of the backend span is downstream work
PostgreSQL INSERT	~0.81 ms	Negligible for this request
Resend POST	~161.8 ms	Dominant latency source
PostgreSQL UPDATE	~1.22 ms	Negligible for this request
Total browser trace	~190 ms	Acceptable for a simple contact submission

DECISION Do not optimize what the trace says is not expensive. PostgreSQL is not the bottleneck here. The synchronous Resend request is.

The next performance question, if it ever matters, is asynchronous email delivery: queue the email, return sooner, and process the external call outside the browser request. That would be a real reason to consider an asynchronous component such as Redis or another queueing system later.

Security, boundaries, and debugging lessons

V3 introduced more secrets and more public endpoints, so configuration boundaries matter.

Secret / config	Location	Why
PostgreSQL password	gitignored backend/.env	Database authentication
Resend API key	gitignored backend/.env	Email provider authentication
Grafana Alloy token	gitignored alloy/.env	Telemetry export authentication
Deployment write token	gitignored environment value	Protect POST /api/deployments
Frontend config	Next.js environment variables	Browser-visible values are not secrets

One useful debugging lesson came from PostgreSQL 18 itself: the container image changed its preferred data-directory mount convention. The old path produced a real startup error; the fix came from reading the container's actual error output rather than assuming older Docker examples still applied.

Another lesson came from the Faro side: the remaining internalLogger console error is an upstream SDK failure-path bug triggered when a real browser telemetry send fails. The local application code was tested for initialization regressions and left unchanged; suppressing the symptom would have hidden the real behavior.

RULE FOR THE LAB Use the observed request, response, container log, trace, or source behavior as evidence. Do not “fix” a symptom by guessing.

What V3 deliberately did not build

Deferred technology	Why it stays deferred
Redis	No current cache or queue problem worth adding another stateful system for
Kafka	No current event-processing requirement outside the request/response cycle
Database Observability	Useful later for deeper PostgreSQL health/query monitoring; application tracing already covers request-level DB spans
AWS production hosting	Required before real public visitors can generate production backend/DB analytics
GitHub Actions deployment recording	Best added when there is a reachable production backend; avoid a temporary self-hosted runner

The sequence is intentional: use PostgreSQL until it creates a real problem, then add the technology that solves that problem. This avoids building infrastructure merely because it appears on a technology checklist.

The next large architectural phase is productionizing the backend/data layer on AWS. That phase will turn the local design into a system reachable by real visitors and will make the analytics features meaningful outside the developer workstation.

What V3 taught

V3 changed the application from a stateless API lab into a small data-backed system. The important mental models are:

1. Tables are durable facts. Each row is a piece of application history that can survive process/container restarts.
2. Connections are a resource. A pool lets request traffic reuse database sessions rather than recreating them repeatedly.
3. Transactions have boundaries. PostgreSQL can guarantee its own writes, but an external HTTP provider such as Resend is a separate system.
4. Tracing explains request behavior. Database spans let us compare database time with downstream service time on the same request.
5. Public write endpoints need intentional boundaries. Anonymous analytics can be public because they store low-sensitivity counters; deployment writes are protected because they mutate operational history.

Useful local commands

```
docker compose up -d --build
curl http://localhost:8000/health
curl http://localhost:8000/health/db
```

```
# Inspect PostgreSQL locally
psql postgresql://<user>@localhost:5432/<db>
```

```
# Inspect container state
docker compose ps
docker compose logs backend
docker compose logs postgres
```

WORKING RHYTHM Build -> Test locally -> Observe -> Verify -> Document -> Commit -> Push. Add the next system only when the application has a real reason for it.

V3 completion record

Check	Result
Contact history	PASS - real submission persisted
Resume analytics	PASS - PDF/DOCX events recorded
Project analytics	PASS - project events recorded and aggregated
Deployment history	PASS - real Git history seeded; protected write endpoint exists
PostgreSQL persistence	PASS - survived restart and container recreation
Connection pooling	PASS - configured and self-healed after stale connections
PostgreSQL OTel spans	PASS - INSERT/UPDATE/SELECT visible in traces
Browser -> backend tracing	PASS - matching trace IDs
Alloy export	PASS - successful export with zero failures during final validation
Application Observability	PASS - backend and frontend represented as services during validation
Website / README	PASS - V3 local/prod boundaries updated
Repository hygiene	PASS - secrets excluded from source control

V3 STATUS Complete. PostgreSQL is now a real application data layer in the local lab. FastAPI, Alloy, and PostgreSQL remain local-only until production infrastructure is added.

V3 / LEARNING GUIDE

PostgreSQL gave the application memory. The next phase is to decide where that application should live in production.